

Sprawozdanie z Algorytmów Ewolucyjnych

Problem cVRP

Adrian Sośniak 259148

Październik 2025

Contents

1	Cel sprawozdania	2
2	Opis problemu CVRP	2
2.1	Dane wejściowe	2
2.2	Reprezentacja problemu	2
2.3	Reprezentacja problemu/osobnika	3
3	Implementacja symulowanego wyżarzania	3
3.1	Inicjalizacja rozwiązania	4
3.2	Generowanie sąsiedztwa	4
3.3	Akceptacja rozwiązań	4
3.4	Chłodzenie	4
4	Implementacja algorytmu ewolucyjnego	4
4.1	Inicjalizacja populacji	5
4.2	Ocena osobnika	5
4.3	Selekcja	5
4.4	Krzyżowanie	5
4.5	Mutacja	6
5	Eksperymenty i analiza wyników	7
5.1	Konfiguracja eksperymentów	7
6	Wnioski	10

1 Cel sprawozdania

Celem tego sprawozdania jest porównanie metaheurestyk oraz heurestyk w obszarze problemu CVRP (z ang. Capacitated Vehicle Routing Problem) z głównym naciskiem na metody ewolucyjne. Porównano metode zachłanna, losowa oraz algorytm ewolucyjny skupiając się na jakości wyszukiwanych rozwiązań.

2 Opis problemu CVRP

CVRP (z ang. Capacitated Vehicle Routing Problem) jest problemem który w swojej naturze przypomina TSP (z ang. Traveling Salesman Problem).

TSP polega na wyznaczeniu najkrótszej trasy przechodzącej przez wszystkie lokalizacje najwyżej raz. CVRP różni się od niego tym, że każde miasto posiada danej wagi ładunek który pojazd zabiera podczas wizyty, pojemność która nie pozwala zabierać ładunków które przekrocza limit wagi załadunku, oraz specjalna lokalizacja, magazyn, w którym pojazd pozbywa się ładunków, oraz w którym musi zacząć oraz skończyć swoją podróż. Do magazynu pojazd może wracać dowolna ilość razy.

Przez to iż sam TSP jest złożoności $O(n!)$ a CVRP jest problemem który dodaje do TSP jeszcze więcej możliwości (takich jak powrót do magazynu) czyni go problemem NP-trudnym, czyli nieefektywnym dla metod takich jak przegląd zupełny.

2.1 Dane wejściowe

Do testów wykorzystano pliki typu .vrp. Jest to format pliku który standaryzuje formę opisu problemu. W plikach z których korzystam w tym sprawozdaniu lokalizacje są opisywane przez ich koordynaty w dwóch wymiarach w przestrzeni euklidesowej. Lokalizacja o indeksie 0 jest uznawana jako magazyn. Zawiera on też kluczowe informacje o rozmiarze problemu (DIMENSION), pojemność (CAPACITY) oraz listę wag ładunków dla każdej lokalizacji (DEMAND SECTION).

2.2 Reprezentacja problemu

Na potrzeby realizacji programu lokalizacje i ich koordynaty zostały przekonwertowane na macierz odległości która po przeliczeniu, pozwala na szybki dostęp do długości pomiędzy wybranymi lokalizacjami.

2.3 Reprezentacja problemu/osobnika

By móc ewaluować rozwiązania jest potrzebna jakaś forma ich reprezentacji. Na potrzeby tego sprawozdania autor zdecydował o reprezentacji problemu w formie wektora liczb całkowitych o rozmiarze DIMENSION-1 w którym zawarty są indeksy każdej z lokalizacji nie licząc startowej (czyli 0).

Kolejność elementów w tym wektorze decyduje o kolejności odwiedzanych lokalizacji, stosując wymuszony powrót do magazynu gdy następna odwiedzana lokalizacja posiada ładunek o wadze większej niż jest to możliwe dla pojazdu na dany moment do przewiezienia.

Istnieją również alternatywy, takie jak rozszerzenie wektora o dodatkowe zera lub unikalne wartości ujemne, które gwarantują reprezentację optymalnego rozwiązania, jednak z powodu tego iż poprzedni sposób znacznie zmniejsza rozmiar wektora, oraz poszukujemy rozwiązania głównie efektywnego, porzucono te formy reprezentacji.

3 Implementacja symulowanego wyżarzania

Algorytm symulowanego wyżarzania (SA, ang. *Simulated Annealing*) został zaimplementowany jako uniwersalny moduł umożliwiający jego zastosowanie do dowolnego problemu optymalizacyjnego, w tym do problemu CVRP. Dzięki odpowiedniej konstrukcji klasy algorytmu możliwe jest zewnętrzne ustalenie temperatury początkowej, tempa chłodzenia oraz liczby iteracji wykonywanych dla danej temperatury.

W przeciwieństwie do algorytmu ewolucyjnego, który operuje na populacji rozwiązań, symulowane wyżarzanie utrzymuje i modyfikuje jedno aktualne rozwiązanie. Jego istota jest możliwość akceptacji gorszych rozwiązań z pewnym prawdopodobieństwem, co pozwala uniknąć przedwczesnego zbiegania do minimum lokalnego.

```
// === Main methods ===
void Init();
void Iterate();
void IterateWithLogging(SolutionsLogger& logger, int logInterval = 1000);

// === SA Parameters ===
double startTemp = 1000.0;
double minTemp = 1e-3;
double alpha = 0.995;
int iterationsPerTemp = 1000;
int maxNoImprove = 2000;
```

Figure 1: Kluczowy kod dla algorytmu symulowanego wyżarzania

3.1 Inicjalizacja rozwiązania

Algorytm rozpoczyna działanie od wygenerowania losowego rozwiązania. Następnie obliczana jest jego wartość funkcji celu przy pomocy metody oceny wykorzystywanej w problemie CVRP. Wartość ta stanowi punkt odniesienia dla dalszej optymalizacji. Równocześnie tworzona jest kopia najlepszego dotychczas znalezionego rozwiązania.

3.2 Generowanie sąsiedztwa

Dla bieżącego rozwiązania generowane jest sąsiedztwo poprzez prostą operację *swap* — zamianę miejscami dwóch losowo wybranych elementów w reprezentacji trasy. Takie podejście pozwala na zachowanie prostoty implementacji przy jednoczesnym zapewnieniu różnorodności generowanych kandydatów.

3.3 Akceptacja rozwiązań

Nowe rozwiązanie jest przyjmowane, jeśli jego wartość funkcji celu jest lepsza (mniejsza) niż dotychczasowa. W przeciwnym przypadku może zostać zaakceptowane z pewnym prawdopodobieństwem określonym wzorem:

$$P = e^{\frac{-(\text{nowe} - \text{stare})}{T}},$$

gdzie T oznacza aktualną temperaturę. Dzięki temu mechanizmowi algorytm może czasem cofnąć się do gorszego rozwiązania, co zwiększa szansę na znalezienie optimum globalnego.

3.4 Chłodzenie

Po wykonaniu ustalonej liczby iteracji dla danej temperatury, temperatura jest zmniejszana zgodnie ze wzorem:

$$T = \alpha \cdot T,$$

gdzie $\alpha \in (0, 1)$ jest współczynnikiem chłodzenia. Proces ten trwa do momentu, aż temperatura osiągnie wartość minimalną lub liczba iteracji bez znalezienia lepszego rozwiązania przekroczy dany próg.

4 Implementacja algorytmu ewolucyjnego

Do realizacji zadania zdecydowano się na stworzenie elastycznego algorytmu ewolucyjnego. Do jego poprawnego działania musimy zdefiniować m.in. funkcje

oceny osobnika, krzyżowania, mutacji oraz tworzenia losowego osobnika. Pozwala to na dostosowanie algorytmu do dowolnego problemu. Obecna implementacja pozwala na zewnętrzne ustalenie warunku stopu. Tu algorytm ewolucyjny zatrzymuje się po wykonaniu ustalonej zewnętrznie liczby iteracji.

```

void EvoAlgorithm::Start()
{
    _population->Init();
    _population->Eval();
}

void EvoAlgorithm::Loop()
{
    //std::cout<< "Select"<<std::endl;
    _population->Select();
    //std::cout<< "Cross"<<std::endl;
    _population->Cross();
    //std::cout<< "Mutate"<<std::endl;
    _population->Mutate();
    //std::cout<< "Eval"<<std::endl;
    _population->Eval();
}

struct PopParameters
{
    int init_population_size = 100;
    int selection_population_size = 30;
    int elitism_count = 1;
    double mutation_chance;
    double crossover_chance;
    void (*mutationOperator)(Solution& s, double mutChance);
    Solution* (*crossOperator)(Solution& s1, Solution& s2);
    Solution* (*randSolutionCreator)();
    double (*evaluator)(Solution& s);
    double BEST_W = 1;
    double WORST_W = 40;
    double interpolation_strength = 1;
    bool useRulate = true;
};

```

Figure 2: Kluczowy kod dla algorytmu ewolucyjnego

4.1 Inicjalizacja populacji

Populacja jest inicjowana przez stworzenie ustalonej w parametrach liczby początkowych losowych rozwiązań.

4.2 Ocena osobnika

Ocena osobnika realizowana jest poprzez zachłanne przejście po miastach w wektorze.

4.3 Selekcja

Zdecydowano się na ruletkę, ze zmodyfikowanym prawdopodobieństwem przetrwania. Zmiana ta spowodowana była przez to, że prawdopodobieństwo usunięcia osobnika było liniowo zależne od oceny jego rozwiązania, co nie wystarczająco faworyzowało najlepsze rozwiązania. Zmieniono prawdopodobieństwo przez nadanie wartości od min do max w liniowej zależności od pozycji wrankingu. Im lepsze rozwiązanie tym mniejsza wartość nadana.

4.4 Krzyżowanie

Każdy osobnik bierze udział w losowaniu. Przy wygranej wyznaczany jest losowy osobnik do krzyżowania. Po zastosowaniu operatora nowe osobniki dodawane są do populacji. Zaimplementowano OX, CX oraz PMX.

4.5 Mutacja

Tak jak w krzyżowaniu każdy osobnik w populacji bierze udział w losowaniu. Osobniki te są poddawane operacji mutacji i zmienione wracają do populacji. Taka implementacja umożliwia utracenie najlepszego rozwiązania, dlatego ważne jest zapisywanie na bieżąco interesujących nas rozwiązań.

5 Eksperymenty i analiza wyników

Table 1: Porównanie wyników algorytmów dla CVRP

Instancja	Random [10k]					Greedy [n]				EA [10x]				SA [10x]			
	opt	best	worst	avg	std	best	worst	avg	std	best	worst	avg	std	best	worst	avg	std
a-n32-k5	784	1613	2669	2140	142	1087	1328	1219	82	819	987	889	53	803	856	826	17
a-n37-k6	949	1746	2686	2236	133	1263	1578	1431	81	1012	1091	1048	28	971	998	984	10
a-n39-k5	822	1544	2686	2193	130	1105	1565	1277	116	891	1014	962	40	834	857	843	9
a-n45-k6	944	2175	3356	2831	152	1401	1796	1546	95	1056	1147	1098	32	967	1009	987	13
a-n48-k7	1073	2368	3657	2979	173	1477	1738	1607	67	1173	1319	1244	38	1094	1167	1129	20
a-n54-k7	1167	2653	3937	3308	172	1626	1946	1812	74	1313	1427	1371	36	1187	1253	1211	21
a-n60-k9	1354	3042	4348	3721	177	1884	2355	2094	111	1516	1775	1626	72	1374	1451	1406	24

5.1 Konfiguracja eksperymentów

Do eksperymentów na algorytmie ewolucyjnym wykorzystano następujące parametry

- Populacja startowa (init_pop_size) - 1000
- Liczba pokoleń -1500
- Prawdopodobieństwo krzyżowania (Px) - 0.7
- Prawdopodobieństwo mutacji (Pm) - 0.1
- Populacja docelowa (selection_pop_size) - 200/300/400 w zależności od trudności problemu
- Operator krzyżowania/mutacji - wykorzystano cycle crossover oraz w przypadku mutacji inverse
- Elitaryzm - zachowuje 25% najlepszych osobników
- Ruletki - zmodyfikowana ruletka, najlepsza wartość to 1 najgorsza to 40

Do eksperymentów na symulowanym wyżarzaniu wykorzystano następujące parametry

- Temperatura początkowa (init_pop_size) - 1000
- Temperatura końcowa -0.001
- Współczynnik chłodzenia (alpha) - 0.95
- Ilość iteracji dla danej temperatury - 10000
- Ilość iteracji bez poprawy - 500000

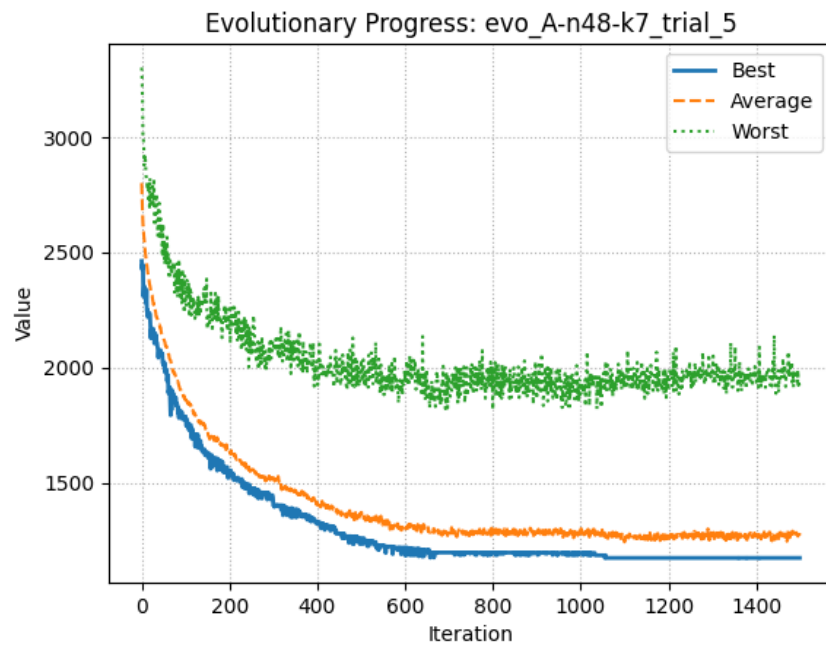


Figure 3: Przebieg zmian wartości rozwiązania dla 5 próby instancji A-n48-k7 (EVO).

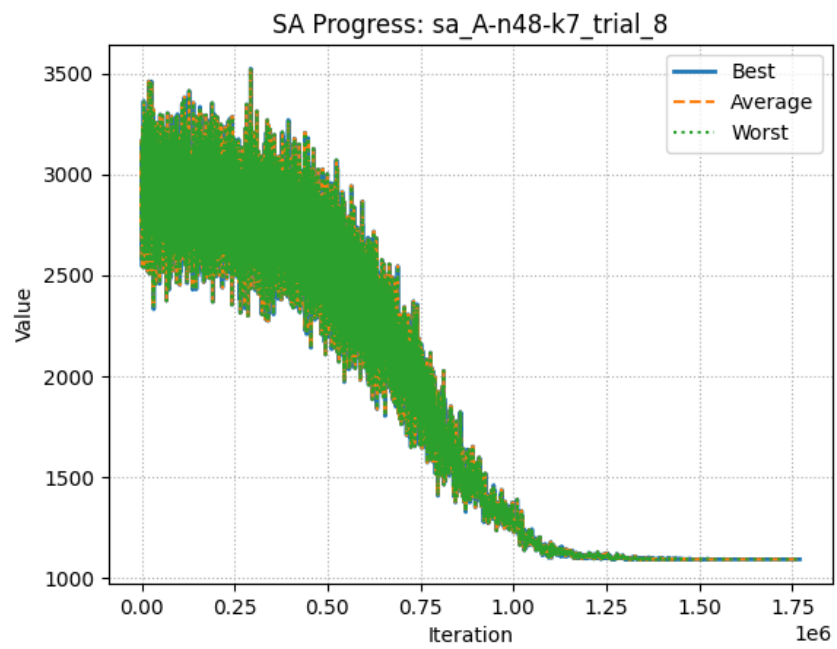


Figure 4: Przebieg zmian wartości rozwiązania dla 8 próby instancji A-n48-k7 (SA).

6 Wnioski

Z wyników widać, że oba metaheurestyki wypadły lepiej nad metoda losowa jak i zachłanna.

Po średnich i odchyleniach standardowych widać że SA produkuje konsekwentnie rozwiązania skuteczne, trochę gorzej za to w tym wypadł algorytm ewolucyjny. Mimo tego żadnen z końcowych wyników nie wypadł gorzej od najlepszego z metody "greedy".

Przez sposób reprezentacji genomu możliwe jest to że zarówno SA jak i alg. ewolucyjny mogły nie znaleźć optymalnego rozwiązania, co może tłumaczyć tak bliskie jednak nie optymalne wyniki SA.